

Reviewer Pack — Persistent Homology Barcode Length Inversely Proportional to...

Entry 42bdd22c09b4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 19:41:26 UTC

Persistent Homology Barcode Length Inversely Proportional to DPLL Tree Size

Entry ID: 42bdd22c09b4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 19:41:26 UTC

1. Conjecture

Field A (mathematical branch): Persistent Homology **Field B** (complexity object): DPLL Search Tree

Statement:

For 3-SAT instances with m clauses and n variables, the total barcode length L of the persistent homology of the clique complex built from clause-sharing edges satisfies $L = \Theta(1 / T_{\text{DPLL}})$, where T_{DPLL} is the DPLL tree size. For all instances with $n \leq 40$, $|L * T_{\text{DPLL}}| \leq 1 + \varepsilon$ for $\varepsilon < 0.1$.

Rationale (proposer's reasoning):

Persistent homology captures topological structure of clause interactions, which may correlate with search space complexity. Higher connectivity (persistent features) could imply more complex landscapes, inversely affecting DPLL efficiency via branching factors.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d98d4ada15d16f57

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def dpll(clauses, assignment={}):
    if not clauses:
        return True
    literal = next(l for l in range(1, len(assignment) + 2) if l not in assignment and -l not in assignment)
    positive_clauses = [c for c in clauses if literal in c]
    negative_clauses = [c for c in clauses if -literal in c]
    if dpll(positive_clauses, {**assignment, literal: True}):
        return True
    elif dpll(negative_clauses, {**assignment, literal: False}):
        return True
    else:
        return False

def build_clique_complex(clauses):
    n = max(abs(l) for l in set.union(*clauses))
    adjacency_matrix = [[0] * (n + 1) for _ in range(n + 1)]
    for clause in clauses:
        for i, j in combinations(sorted(abs(l) for l in clause), 2):
            adjacency_matrix[i][j] = 1
            adjacency_matrix[j][i] = 1
    return adjacency_matrix

def compute_persistent_homology_barcode_length(adjacency_matrix):
    n = len(adjacency_matrix)
    barcode_length = 0
    for i in range(1, n + 1):
        row_sum = sum(adjacency_matrix[i])
        if row_sum == i - 1:
            barcode_length += 1
    return barcode_length

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    m = random.randint(n * 2, n * 3)
```

```

clauses = []
for _ in range(m):
    clause = [random.choice([-1, 1]) * (i + 1) for i in random.sample(range(n), random.randint(1, n))]
    clauses.append(clause)

adjacency_matrix = build_clique_complex(clauses)
barcode_length = compute_persistent_homology_barcode_length(adjacency_matrix)
dpll_tree_size = len(dpll(clauses))

metric_value = abs(barcode_length * dpll_tree_size - 1)
conjecture_holds = metric_value < 0.1
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "barcode_length_times_dpll_tree_size",
    "metric_value": metric_value,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3**j + 5**k for i, j, k in product(range(5), range(5), range(5))]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_61711205.py", line 79, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_61711205.py", line 57, in run_trial

```

```
adjacency_matrix = build_clique_complex(clauses)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_61711205.py", line 31, in build_clique_
n = max(abs(l) for l in set.union(*clauses))
^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

TypeError: descriptor 'union' for 'set' objects doesn't apply to a 'list' object

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to TypeError in set.union operation on list objects, preventing data collection | next: Fix the build_clique_complex function to properly handle list inputs (e.g., convert to sets first)

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111755
2	propose	ollama_remote	qwen3:8b	0	0	75945
3	preregistration	ollama_remote	qwen3:8b	0	0	24186
4	novelty	ollama_remote	qwen3:8b	0	0	20900
5	novelty	ollama_remote	qwen3:8b	0	0	14890
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18034
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9698
8	judge	ollama_remote	qwen3:8b	0	0	16734

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 292142 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/42bdd22c09b4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/42bdd22c09b4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/42bdd22c09b4.tar.gz` (if generated)